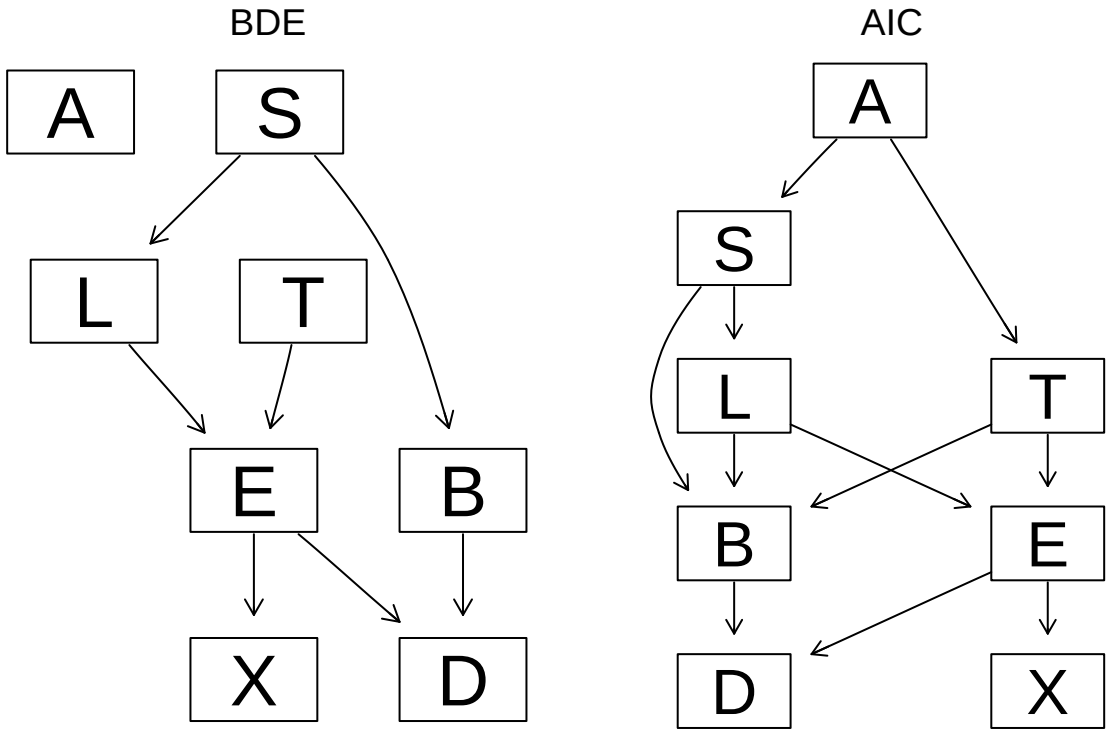
Lab 1: Graphical Model

Manfred Clase (mancl774)

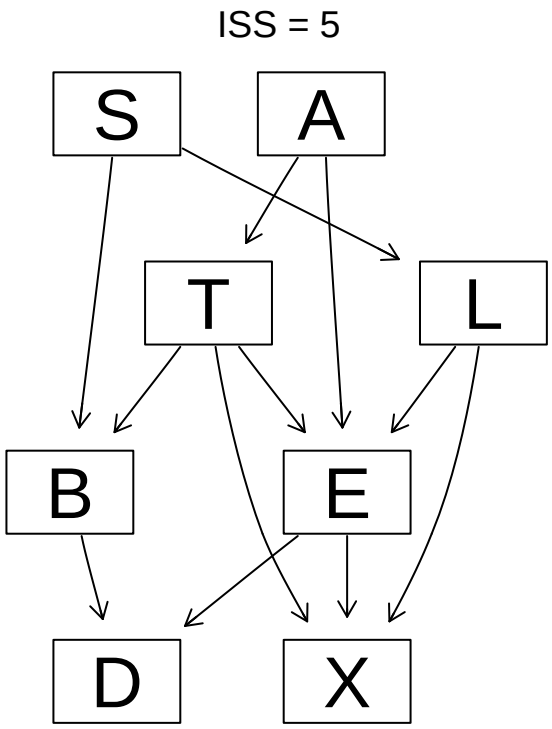
2026-09-13

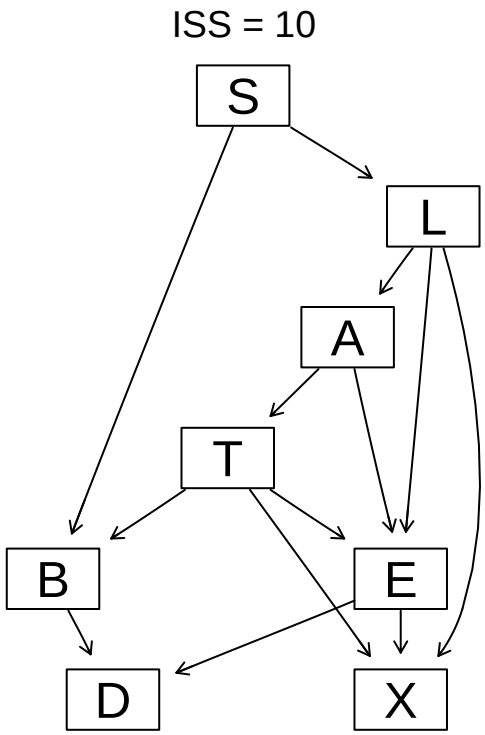
Question 1

Show that multiple runs of the hill-climbing algorithm can return non-equivalent Bayesian network (BN) structures. Explain why this happens.

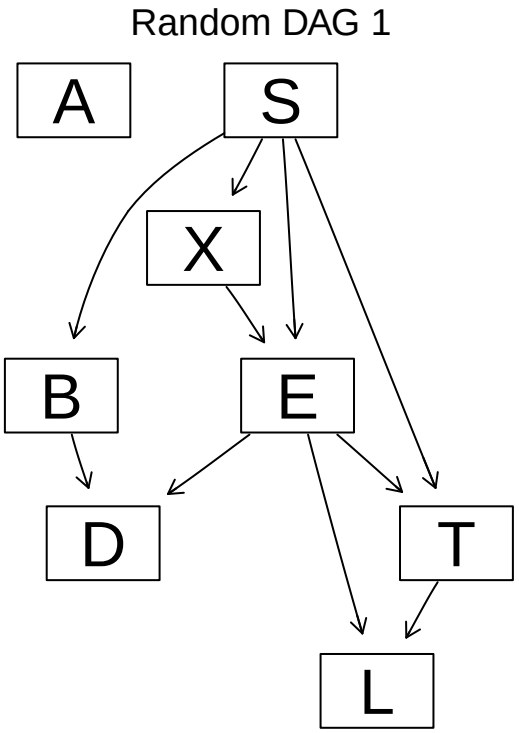


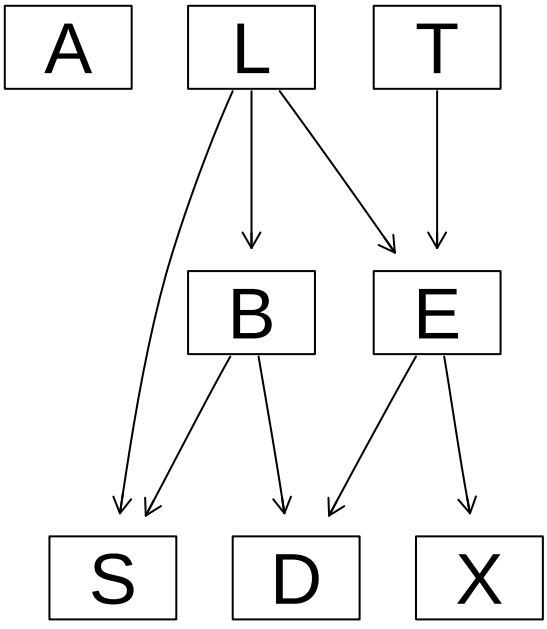

```
## [1] "Different number of directed/undirected arcs"
```






```
## [1] "Different number of directed/undirected arcs"
```






```
## [1] "Different number of directed/undirected arcs"
```

Implementation: Ran hc() on the full Asia data three ways: BDe vs. AIC score, BDe with iss = 5 vs. iss = 10, and BDe from two different random.graph() starting DAGs (seed 12345). Each pair was compared via all.equal() on their cpdag() equivalence classes.

Answer. All three variants of hill climbing (different score function, different imaginary sample size, and different random starting structures; code in Appendix A.1) return structures that are not equivalent. This is shown by the plots and by converting each DAG to its equivalence class with `cpdag()` and comparing with `all.equal()`, which reports a different number of directed/undirected arcs in every case.

Hill climbing is a greedy local search: starting from an initial DAG (empty by default) it repeatedly applies the single arc addition, deletion or reversal that most improves the score, and stops when no change improves it. It therefore returns a local optimum, and which one it reaches depends on the starting structure, the score function, the imaginary sample size, random restarts and the data.

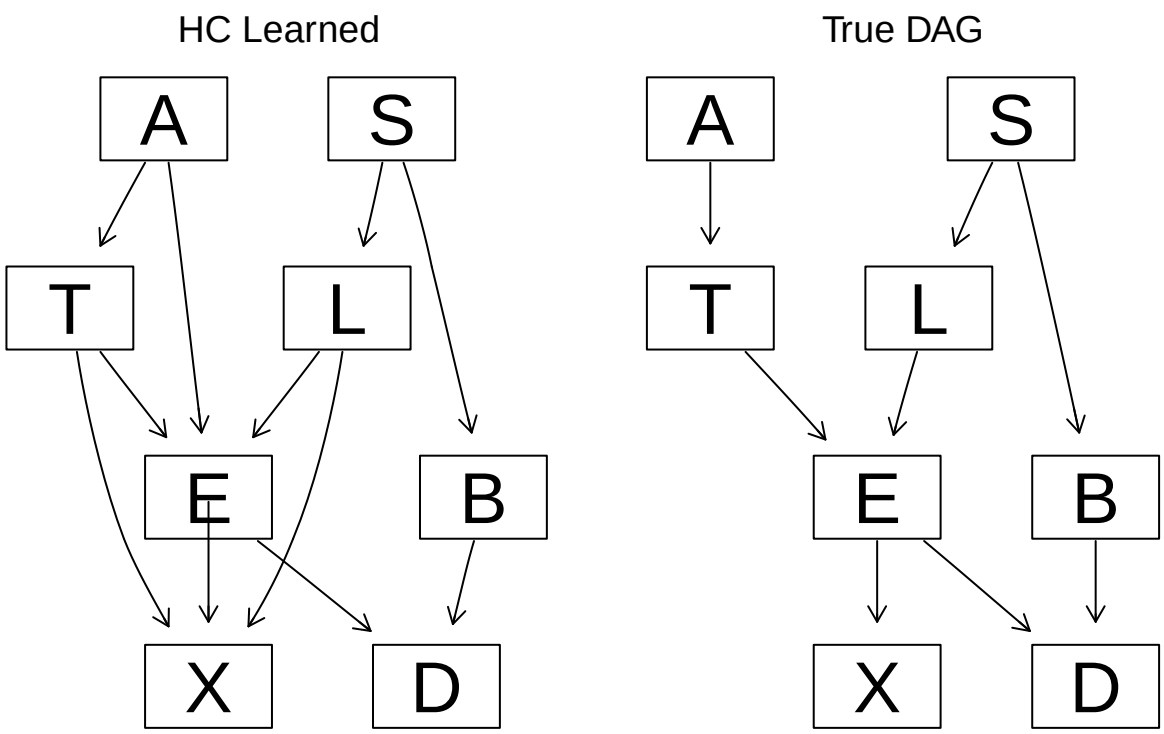
Equivalence. Two DAGs are Markov equivalent iff they have the same skeleton and the same unshielded colliders (v-structures, $X \to Z \leftarrow Y$ with X, Y non-adjacent) (Verma and Pearl). They encode the same conditional independencies and can represent the same distributions, so data alone cannot distinguish them. A class is represented by its CPDAG (directed arcs compelled, undirected arcs reversible), hence comparing `cpdag()`s; `vstructs()` lists the v-structures.

Score equivalence. BDeu (BDe with uniform prior), BIC, AIC and the log-likelihood give equal scores to equivalent DAGs, so the search cannot prefer one member of a class; only differing CPDAGs, as here, are genuinely different models.

Imaginary sample size. `iss` is the total weight of the uniform Dirichlet prior in BDeu. A larger `iss` weakens the regularisation towards the empty graph and typically yields denser graphs. Here iss = 10 gives one more arc than iss = 5, and both are denser than the default iss = 1 graph, which in turn is sparser than the AIC graph.

Question 2

Learn a BN from 80 % of the Asia dataset. Use the BN learned to classify the remaining 20 % of the Asia dataset in two classes: S = yes and S = no.




```
##      hc_model_preds
##        no yes
##   no  337 176
##   yes 121 366
```

```
##      true_model_preds
##        no yes
##   no  337 176
##   yes 121 366
```

Implementation: Split the data 80/20 (seed 12345), learned a DAG with hc() on the training data (BDe, iss = 5) and, alongside the true Asia DAG, fitted both with bn.fit (MLE) and converted with as.grain. Each test row's S was predicted by setting evidence on all other nodes and taking "yes" if querygrain gives $P(S = \text{yes}) > 0.5$.

Answer. A network was learned from the 80 % training split using hill climbing with the BDE score and fitted by maximum likelihood (Appendix A.2). The two confusion matrices are identical, so the learned network and the true network classify every test case the same way, at an accuracy of 0.703.

The posteriors are computed exactly: gRain compiles the BN into a junction tree (moralise, triangulate, form cliques) and propagates evidence by message passing; sampling-based `cpquery`/`cpdist` would only estimate them.

The learned structure is not the true Asia network. What matters for prediction, however, is the posterior $P(S \mid \text{evidence})$, and this depends only on the Markov blanket of S. In both networks $\text{MB}(S) = \{L, B\}$ with $S \to L$ and $S \to B$, so $P(S \mid \text{evidence}) \propto P(S)\,P(L \mid S)\,P(B \mid S)$ with the same MLE estimates from the same training data. The posteriors, and hence the predictions, coincide. Structural differences elsewhere in the graph involve nodes that are conditionally independent of S given the blanket and therefore cannot affect the classification.

Question 3

Now classify S given observations only for the Markov blanket of S.

```
##      hc_mb_preds
##        no yes
##   no  337 176
##   yes 121 366
```

```
##      true_mb_preds
##        no yes
##   no  337 176
##   yes 121 366
```

Implementation: Obtained the Markov blanket of S with mb() from each fitted network (learned and true) and reran the same gRain classifier on the test set, using only those blanket nodes as evidence.

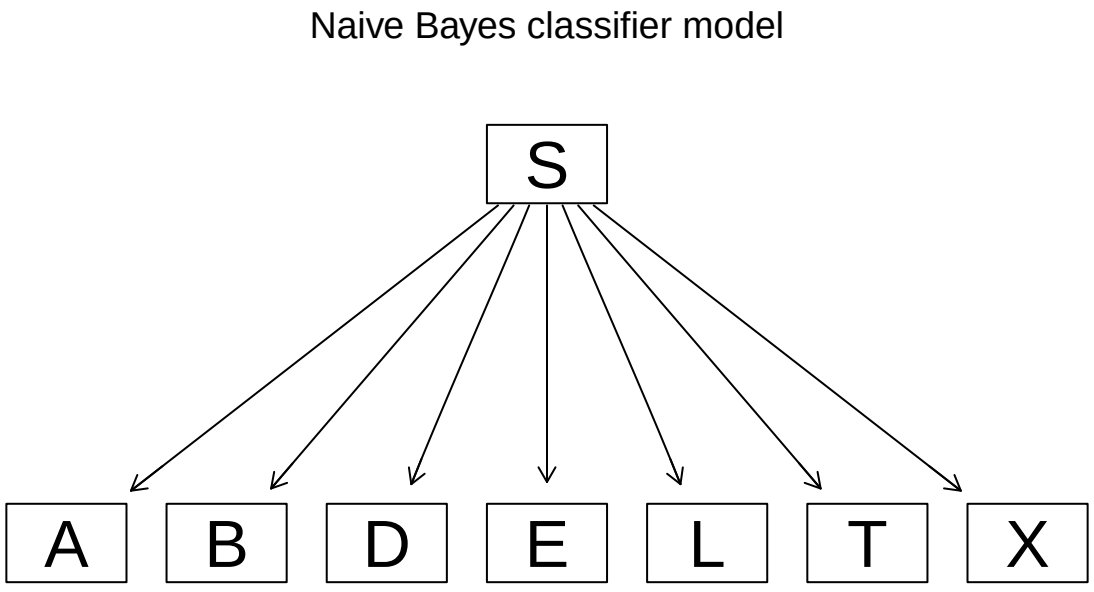
Answer. Restricting the evidence to the Markov blanket of S (Appendix A.3) reproduces the confusion matrices from Question 2 exactly. This is not a coincidence but follows from the definition of a Markov blanket: S is conditionally independent of every other variable given its blanket, so

$$P(S \mid A, T, L, B, E, X, D) = P(S \mid \mathrm{MB}(S)).$$

Observing the remaining variables cannot change the posterior. In d-separation terms, the parents block paths through S's ancestors, the children block paths through its descendants, and the children's other parents are needed because conditioning on a child opens the collider $S \rightarrow C \leftarrow Y$. A practical consequence is that inference requires fewer evidence nodes at no cost in accuracy.

Question 4

Repeat exercise (2) using a naive Bayes classifier.



```
##      nbc_preds
##        no yes
##   no  359 154
##   yes 180 307
```

Implementation: Built the naive Bayes DAG by hand with model2network (S as sole parent of all other nodes), fitted it on the same training split with bn.fit (MLE), converted with as.grain and classified the test set with the same classifier using all non-S variables as evidence.

Answer. The naive Bayes network makes S the sole parent of every other variable, which encodes the assumption that the features are conditionally independent given S (Appendix A.4). The confusion matrix differs from Question 2 and accuracy falls to 0.666.

Note: a BN with own CPTs. When the structure and probabilities are given (not learned), the CPTs can be entered directly with gRain and queried the same way. In `cptable` the values list the child's levels fastest, one parent configuration at a time.

```
# Minimal template: R -> W, with own CPTs
yn <- c("yes", "no")
r <- cptable(~R, values = c(0.2, 0.8), levels = yn)    # P(R)
# P(W | R): (W=yes, W=no) for R=yes, then (W=yes, W=no) for R=no
w <- cptable(~W | R, values = c(0.9, 0.1, 0.1, 0.9), levels = yn)
net <- grain(compileCPT(list(r, w)))   # build + compile junction tree
querygrain(setEvidence(net, nodes = "W", states = "yes"), nodes = "R")
```

```
## $R
## R
##       yes        no
## 0.6923077 0.3076923
```

Question 5

Explain why you obtain the same or different results in the exercises (2–4).

Answer. Exercises 2 and 3 agree because the posterior $P(S \mid \text{evidence})$ is determined entirely by the Markov blanket of S. The learned and the true DAG differ in several arcs, but the local structure around S carries the same information, so both give the same posterior; and dropping the evidence that lies outside the blanket removes nothing that the posterior depends on. Recovering the correct graph is therefore not necessary for correct classification, only the blanket around the target variable matters.

Exercise 4 differs because the Naive Bayes classifier is making the assumption that all features are conditionally independent given S, i.e. $P(S \mid x) \propto P(S) \prod_i P(x_i \mid S)$. This is a rather strong assumption that obviously does not hold given the known true DAG. In the Asia domain E is close to a deterministic function of T and L, X depends on the patient only through E, and D depends on B and E. These variables therefore carry largely overlapping evidence. Naive Bayes multiplies their likelihoods as though they were independent sources, which counts the same information several times and produces distorted, over-confident posteriors. Moreover, in naive Bayes every feature is in the Markov blanket of S, so A, T, E, X and D, which are irrelevant given $\{L, B\}$ in the true network, now influence the prediction. The effect is visible in the error pattern and not only in the error rate: naive Bayes predicts "no" more often, gaining correct negatives but losing considerably more on false negatives.

Appendix A: R Code

A.1 Question 1: Hill climbing with variations

```r
# Q1: hill climbing (hc) is a greedy LOCAL search over DAGs, so the
# result depends on score, prior (iss) and start. Compare results by
# equivalence class: cpdag() keeps skeleton + v-structures; all.equal()
# gives TRUE if equivalent, else a description of the differences.

# Different score functions
# score = "bde" | "bic" (default) | "aic" | "loglik" | "k2" ...
dag1 <- hc(asia, score = "bde")
dag2 <- hc(asia, score = "aic")
# graphviz.plot(): draws a bn DAG via Rgraphviz (returns graphNEL invisibly)
graphviz.plot(dag1, main = "BDE")
graphviz.plot(dag2, main = "AIC")
all.equal(cpdag(dag1), cpdag(dag2))

# Different imaginary sample sizes
# iss = pseudo-count weight of the BDe Dirichlet prior (only for bde).
# Larger iss -> smaller penalty on extra arcs -> denser graphs.
dag3 <- hc(asia, score = "bde", iss = 5)
dag4 <- hc(asia, score = "bde", iss = 10)
all.equal(cpdag(dag3), cpdag(dag4))

# Different random starting structures
# start = initial DAG (default empty). Different starts -> different
# local optima. Also useful: restart = k random restarts, perturb = n.
set.seed(12345)
# random.graph(nodes): random DAG (bn object) over the given node names
dag5 <- hc(asia, score = "bde", start = random.graph(colnames(asia)))
dag6 <- hc(asia, score = "bde", start = random.graph(colnames(asia)))
all.equal(cpdag(dag5), cpdag(dag6))
```

A.2 Question 2: Learning, fitting and classifying

```r
# Q2 pipeline: learn DAG (hc) -> fit CPTs (bn.fit) -> gRain junction tree
# (as.grain) -> exact inference (setEvidence + querygrain) -> classify S.

# Split the data into 80% training and 20% test
n <- dim(asia)[1]
set.seed(12345)
id    <- sample(1:n, floor(n * 0.8))
train <- asia[id, ]
test  <- asia[-id, ]

# Learn a structure from the training data and compare with the true DAG
# model2network syntax: "[child|parent1:parent2]", "[X]" = root node.
hc_dag   <- hc(train, score = "bde", iss = 5)
true_dag <- model2network("[A][S][T|A][L|S][B|S][D|B:E][E|T:L][X|E]")
```

```r
# Fit parameters by maximum likelihood
# method = "mle" (relative counts) or "bayes" (Dirichlet posterior, iss=;
# avoids zero probs for unseen configurations). fit$S gives S's CPT.
hc_fit   <- bn.fit(hc_dag,   data = train, method = "mle")
true_fit <- bn.fit(true_dag, data = train, method = "mle")

# Convert to gRain objects for exact inference
# as.grain builds the junction tree (moralise -> triangulate -> cliques).
# If needed: compile(as.grain(fit)).
hc_grain   <- as.grain(hc_fit)
true_grain <- as.grain(true_fit)

# Classify the test set: S = "yes" if P(S = yes | evidence) > 0.5, else "no"
# model: compiled gRain object; observed_nodes: names used as evidence.
# Reads the global `test`; returns a character vector of predictions.
classifier <- function(model, observed_nodes) {
  preds <- character(nrow(test))
  for (i in 1:nrow(test)) {
    # Row values as character (setEvidence wants state names, not factors)
    states   <- as.matrix(test[i, observed_nodes])
    # nodes = variable names, states = their observed values (same order)
    evidence <- setEvidence(model, nodes = observed_nodes, states = states)
    # Posterior marginal of S: named vector c(no = p0, yes = p1).
    # (type = "marginal" default; "joint" for joint over several nodes)
    post     <- querygrain(evidence, nodes = "S")$S
    preds[i] <- ifelse(post["yes"] > 0.5, "yes", "no")
  }
  preds
}

# Evidence = every variable except the class S
nodes <- setdiff(colnames(asia), "S")

hc_model_preds   <- classifier(hc_grain,   nodes)
true_model_preds <- classifier(true_grain, nodes)
# Confusion matrices: rows = true S, columns = predicted S
table(test$S, hc_model_preds)
table(test$S, true_model_preds)
```

A.3 Question 3: Classification with the Markov blanket only

```r
# Markov blanket MB(S) = parents + children + children's other parents.
# S is cond. independent of all other nodes given MB(S), so
# P(S | all) = P(S | MB(S)) -> identical predictions to Q2.
# mb() works on both bn (DAG) and bn.fit objects; returns node names.
hc_mb   <- mb(hc_fit,   node = "S")
true_mb <- mb(true_fit, node = "S")

hc_mb_preds   <- classifier(hc_grain,   hc_mb)
true_mb_preds <- classifier(true_grain, true_mb)
table(test$S, hc_mb_preds)
```

```r
table(test$S, true_mb_preds)
```

A.4 Question 4: Naive Bayes classifier

```r
# The naive Bayes classifier assumes that all feature variables are
# conditionally independent given the class variable S. As a Bayesian network
# this is a structure where S is the single parent of all feature nodes and
# no edges exist between the features.
# (Alternative: bnlearn::naive.bayes(train, "S") builds the same DAG.)
nbc_network <- model2network("[S][A|S][T|S][L|S][B|S][E|S][X|S][D|S]")
graphviz.plot(nbc_network, main = "Naive Bayes classifier model")

# Same pipeline as Q2: fit CPTs -> gRain -> classify using all features
nbc_fit   <- bn.fit(nbc_network, data = train, method = "mle")
nbc_grain <- as.grain(nbc_fit)
nbc_preds <- classifier(nbc_grain, nodes)
table(test$S, nbc_preds)
```